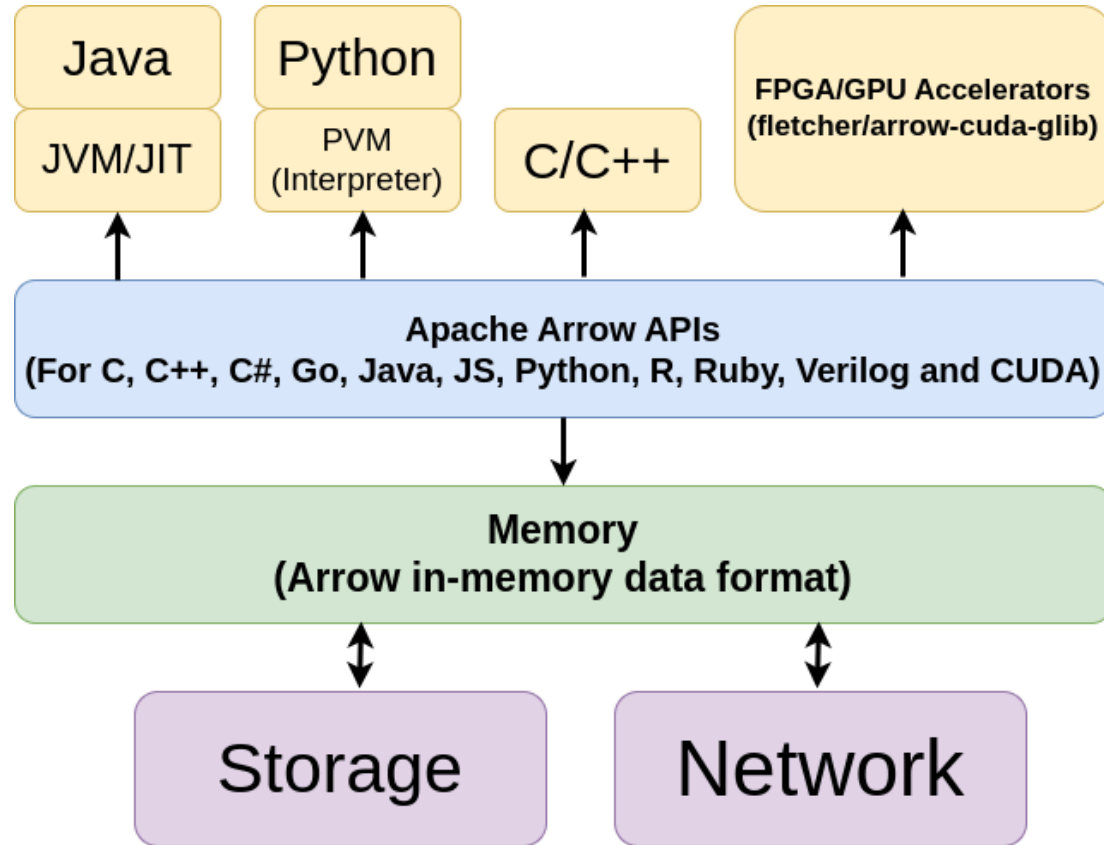


Apache Arrow

Hermilo

17 de Febrero de 2026

¿Qué es Apache Arrow?



- Estandar de intercambio de datos.
- Un formato in-memory.
- Un formato de Red.
- Un formato de Almacenamiento.
- Biblioteca de I/O
- Biblioteca de cómputo vectorizado.
- Biblioteca de manejo de dataframes.
- Un query engine.
- Administrador de datasets particionados.

- El proyecto **Apache Arrow** es un gran esfuerzo dirigido a resolver los problemas fundamentales de la analítica de datos.
- Está dirigido a proporcionar una experiencia **write everywhere, run everywhere**, por lo que es fácil perderse si nos se sabe dónde empezar.
- **PyArrow** es el punto de entrada al ecosistema de Apache Arrow para desarrolladas de Python, y es el medio que ofrece el acceso a los muchos beneficios de Arrow.

PyArrow

- Apache Arrow nació como un **Formato Columnar de Datos**.
- De manera que el tipo fundamental de datos en PyArrow es una **columna de datos** la cual es expuesta por medio de un objeto `pyarrow.Array`.
- A este nivel, PyArrow es similar a los arreglos 1D de **NumPy**.

¿Qué es Columnar?

	ARCHER	LOCATION	YEAR
ROW 1	Legolas	Mirkwood	1954
ROW 2	Oliver	Star City	1941
ROW 3	Merida	Scotland	2012
ROW 4	Lara	London	1996
ROW 5	Artemis	Greece	-600

Row 1	Legolas		Legolas
	Mirkwood		Oliver
	1954	archer	Merida
Row 2	Oliver		Lara
	Star City		Artemis
	1941		
Row 3	Merida	location	Mirkwood
	Scotland		Star City
	2012		Scotland
			London
Row 4	Lara		Greece
	London		
	1996	year	1954
			1941
Row 5	Artemis		2012
	Greece		1996
	-600		-600

Traditional Memory Buffer Arrow Columnar Memory Buffer

¿Qué es **Columnar**?

Con el formato columnar se aprovecha :

- Localidad de Memoria.
 - I/O
 - Vectorización
- Consulta : **Todos los arqueros en Europa**, solo se necesitan dos columnas!. Menos operaciones de I/O, menos uso de memoria.
 - Consulta : **Calcula la media para la columna Year**, solo se necesita una columna!
 - Vectorizar operaciones requiere memoria contigua.
 - La columna ya se encuentra en memoria contigua!

Apache Parquet, a columnar storage format

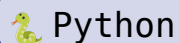


Parquet, a columnar storage format

- Parquet almacena los datos en columnas en lugar de en filas, a diferencia de los formatos de almacenamiento tradicionales basados en filas (como CSV o JSON).
- Este diseño columnar implica que, cuando una consulta solo necesita acceder a unos pocos campos o columnas específicos de un conjunto de datos, Parquet puede leer únicamente las columnas relevantes sin cargar la fila completa en la memoria.
- Parquet también emplea diversas técnicas para mejorar aún más el rendimiento y reducir los costes de almacenamiento:
 - Compression
 - Encoding
 - Metadata and statistics

PyArrow Arrays

```
1  import pyarrow as pa
2
3  ## Arrays can be made of numbers
4  >>> pa.array([1, 2, 3, 4, 5])
5
6  ## Or strings
7  >>> pa.array(["A", "B", "C", "D", "E"])
8
9  ## Or even complex objects
10 >>> pa.array([{"a" : 5}, {"b" : 10}])
11
12 ## Arrays can also be masked
13 >>> pa.array([1, 2, 3, 4, 5],
14 mask = pa.array([True, False, True, False,
15 True])) )
```



Python

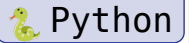
Comparado a los arreglos clásicos de NumPy, los arreglos de PyArrow son un poco más complejos.

- Estandar de intercambio de datos.

Manipulación de Arreglos

- La comunidad Arrow ha creado una implementación de referencia de código abierto de un motor de cálculo y consulta basado en el formato Arrow llamado **Acero**.
- PyArrow proporciona también acceso al motor de cálculo **Acero** mediante el módulo `pyarrow.compute`.
- Para este fin, existe la biblioteca Acero para facilitar diversas implementaciones de alto rendimiento de funciones que operan con datos con formato Arrow, junto con la creación de planes de ejecución de filtrados, agregaciones y transformaciones para flujos de datos.

```
1 import pyarrow.compute as pc
2
3 >>> arr = pa.array([1, 2, 3, 4, 5])
4 >>> pc.multiply(arr, 2)
5 >>> pc.value_counts(arr)
6 >>> pc.min_max(arr)
```



PyArrow Tables

- Como los arreglos son “columnas”, estos pueden ser agrupados para formar `pyarrow.Table`.
- Las tablas son constituidas por `pyarrow.ChunkedArray` de manera que concatenar filas a ellas resultan una operación barata.
- A este nivel, PyArrow es similar a los **DataFrames de Pandas**.

PyArrow Tables

```
1  >>> table = pa.table([
2      pa.array([1, 2, 3, 4, 5]),
3      pa.array(["a", "b", "c", "d", "e"]),
4      pa.array([1.0, 2.0, 3.0, 4.0, 5.0]),
5  ], names = ["col1", "col2", "col3"])
6  >>> table.take([0, 1, 4])
7  col1: [[1,2,5]]
8  col2: [["a","b","e"]]
9  col3: [[1,2,5]]
10 >>> table.schema
11 col1: int64
12 col2: string
13 col3: double
```

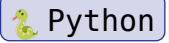


- Comparado a Pandas, las tablas de PyArrow son completamente implementadas en C++ y nunca modifican datos **in-place**.
- Tables están basadas en **ChunkedArrays** de manera que concatenar datos es una operación **zero copy**.
- El motor de cómputo Acero en Arrow es capaz de proporcionar muchas de las operaciones comunes de la analítica de datos como joining, filtrados y agregaciones de datos.

Ejecutando Analítica de Datos

- El motor de cómputo de Acero potencia las capacidades de análisis y transformación disponibles en las tablas
- Muchas funciones de `pyarrow.compute` proporcionan kernels que operan en tablas y las Table exponen métodos de join, filtrado y agrupación.

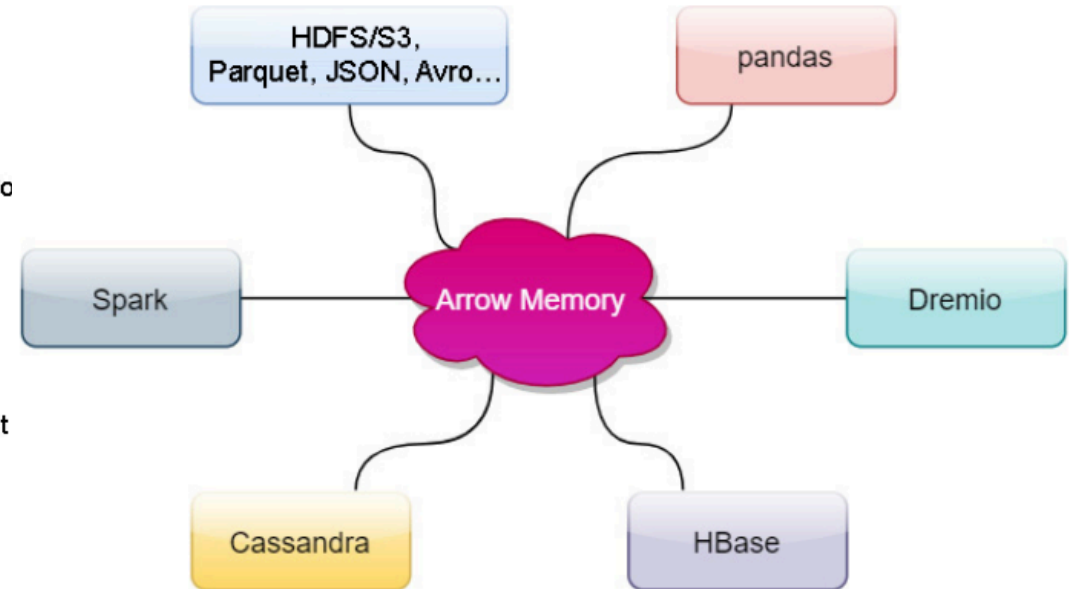
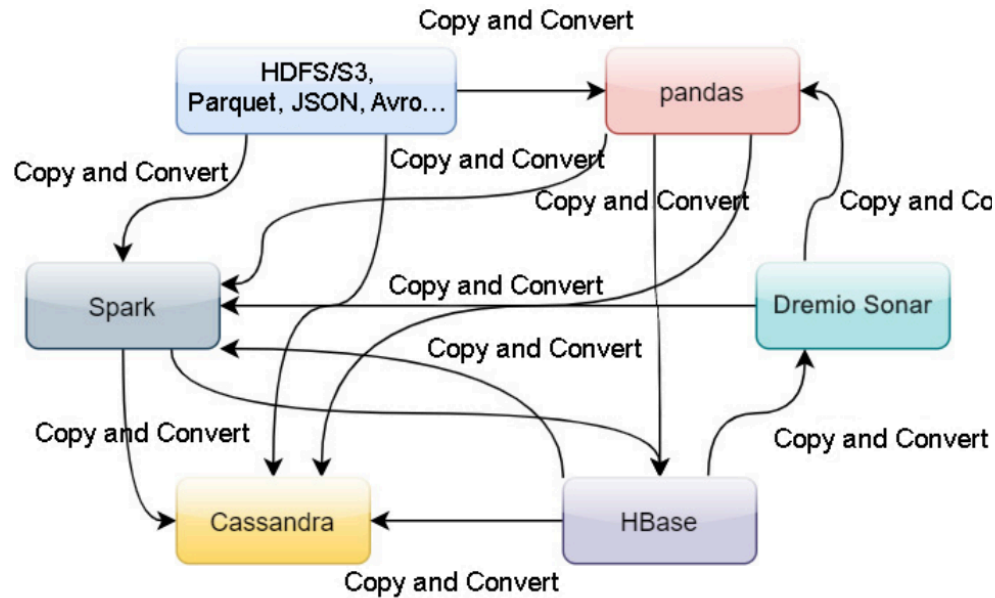
```
1 import pyarrow as pa
2 import pyarrow.compute as pc
3
4 >>> table = pa.table([
5     pa.array(["a", "a", "b", "b", "c", "d", "e", "c"]),
6     pa.array([11, 20, 3, 4, 5, 1, 4, 10])
7 ], names = ["keys", "values"])
8 >>> table.filter(pc.field("values") == )
9 >>> table.groupby("keys").aggregate(["keys", "sum"])
10 >>> table1 = pa.table({"id" : [1, 2, 3],
11     "year" : [2020, 2022, 2019]
12 })
13 >>> table2 = pa.table({"id" : [3, 4],
14     "n_legs" : [5, 100],
15     "animal" : ["Brittle Stars", "Centipede"]
16 })
17 >>> table1.join(table2, keys = "id")
```



PyArrow, NumPy y Pandas

- Uno de las metas de diseño originales de Apache Arrow fue permitir el fácil intercambio de datos sin el costo de convertir a través de múltiples formatos o serializarlos antes de transferirlos.
- PyArrow proporciona el soporte de conversión de datos copy-free de y hacia pandas y numpy.
- Si tenemos datos en PyArrow podemos invocar al método `to_numpy` del objeto `pyarrow.Array` y `to_pandas` en `pyarrow.Array` y `pyarrow.Table` para convertirlos a objetos de pandas o numpy sin enfrentar ningún costo adicional de conversión.

PyArrow, NumPy y Pandas



Tomado de **In-Memory Analytics with Apache Arrow**

Y es rápido!!

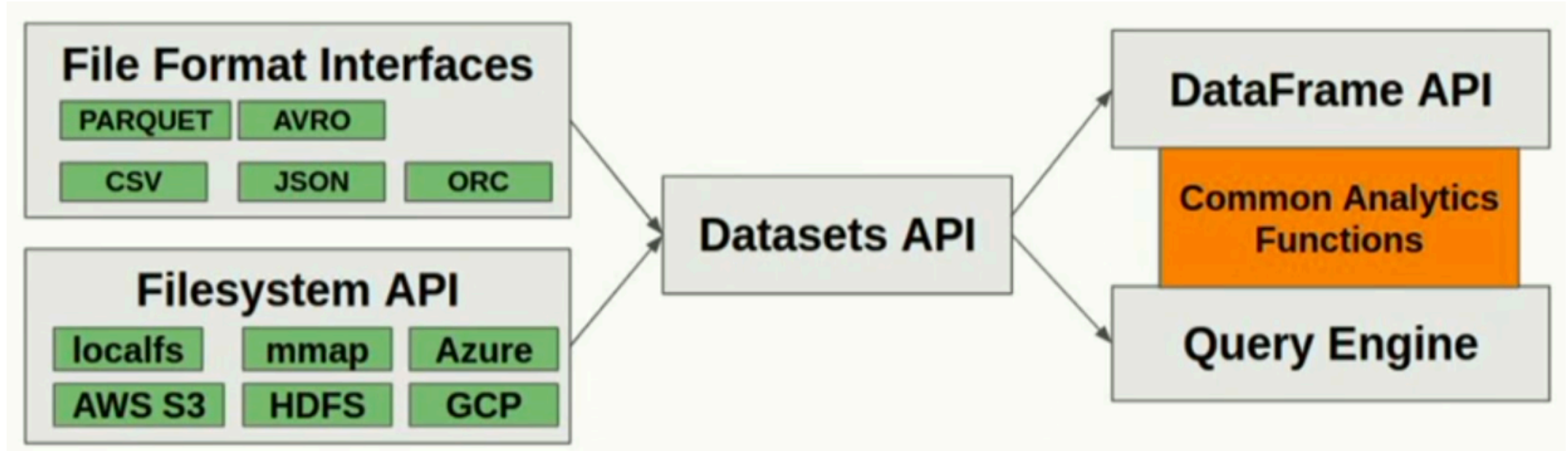
```
1  >>> data = [a % 5 for a in range(100_000_000)]
2  >>> npdata = np.array(data)
3  >>> padata = np.array(data)
4  >>> import timeit
5  >>> timeit.timeit(
6      lambda: np.unique(npdata, return_counts = True),
7      number = 1
8  )
9  >>> timeit.timeit(
10     lambda: pc.value_counts(padata),
11     number=1
12 )
```



Datasets

- **Datasets** son una abstracción que permite trabajar con grandes conjuntos tabulares de datos, potencialmente más grandes que la memoria y distribuidos a través de múltiples archivos.
- Datasets proporciona un acceso **lazy**, evitando la necesidad de cargar todos los datos en memoria inmediatamente.
- Datasets son compatibles con el motor de cómputo Acero en la mayoría de los casos en lugar de las tablas.

Datasets

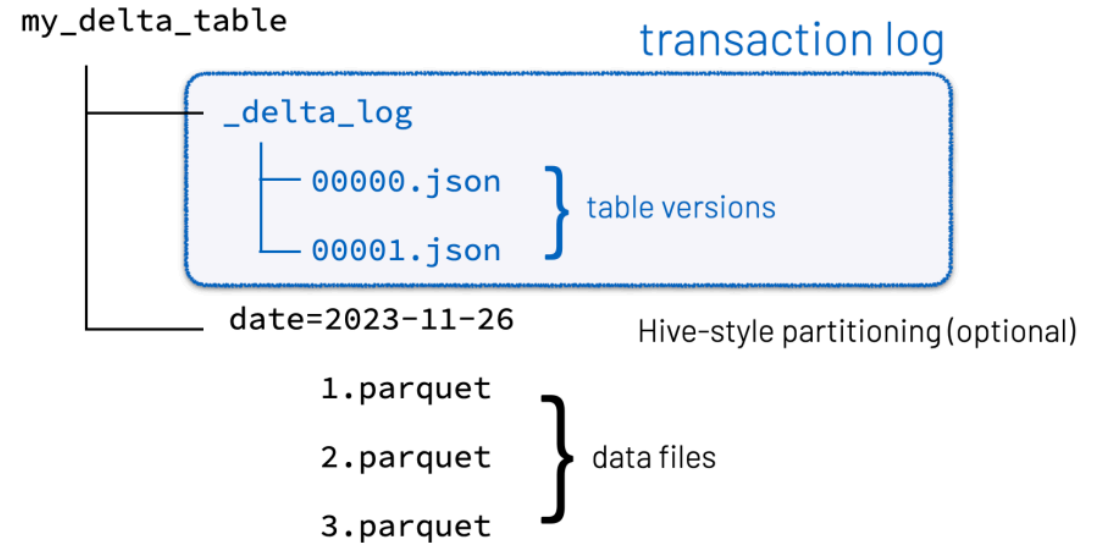


Datasets

- En el actual ecosistema de **data lakes** y **lakehouses**, muchos conjuntos de datos son ahora enormes colecciones de archivos en estructuras de directorios particionados en lugar de un solo archivo.
- La API Datasets proporciona una serie de utilidades para interactuar fácilmente con conjuntos de datos grandes, distribuidos y posiblemente particionados que se distribuyen en múltiples archivos.

Data Lakes : Delta Lake

DELTA LAKE	
metadata (JSON)	data (parquet)
VO - Create Initial Table	
00000.json	Add 1.parquet Add 2.parquet
V1 - Delete Rows	
00001.json	Remove 1.parquet Remove 2.parquet Add 3.parquet

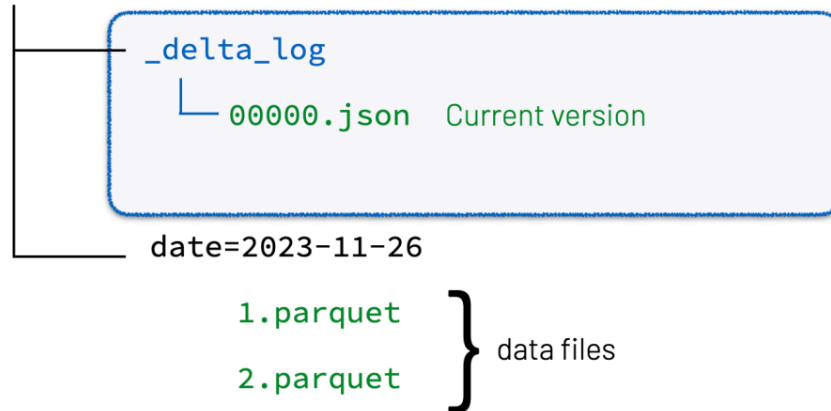


Tomado de Denny Lee, "Understanding the Delta Lake Transaction Log at the File Level", Denny Lee (blog), November 26, 2023.

Data Lakes : Delta Lake

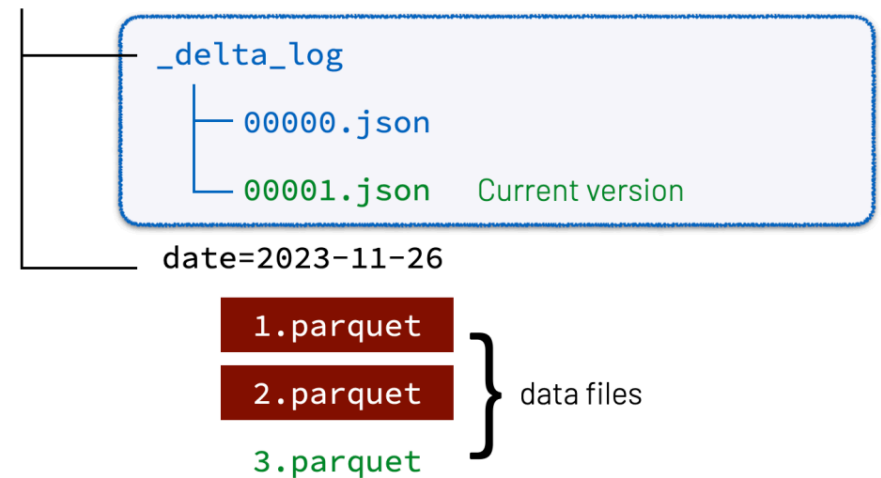
V0 - Create Initial Table

my_delta_table-v0



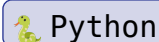
V1 - Delete Rows

my_delta_table-v1



PyArrow Datasets

```
1  >>> from deltalake import DeltaTable
2  >>> import pyarrow.dataset as ds
3
4  # Cargamos la Delta table
5  >>> dt = DeltaTable("datos/atlas")
6
7  # Convertimos a PyArrow Dataset
8  >>> dataset = dt.to_pyarrow_dataset()
9
10 # Observamos los primeros 10 registros
11 >>> dataset.head(10)
12
13 # Filtramos sólo registros de México en 2024
14 >>> filtrado = dataset.scanner(filter=ds.field("year")
    == 2024)#.to_table()
```



- Datasets proporciona acceso lazy a grandes volúmenes de datos guardados en cualquiera de los formatos soportados por PyArrow y accesibles mediante cualquier **FileSystems** reconocido por PyArrow.
- Los Datasets pueden siempre ser convertidos de regreso a Tables para tener acceso al conjunto completo de datos mediante la API de Tables.